

Le Machine Learning pour les nuls

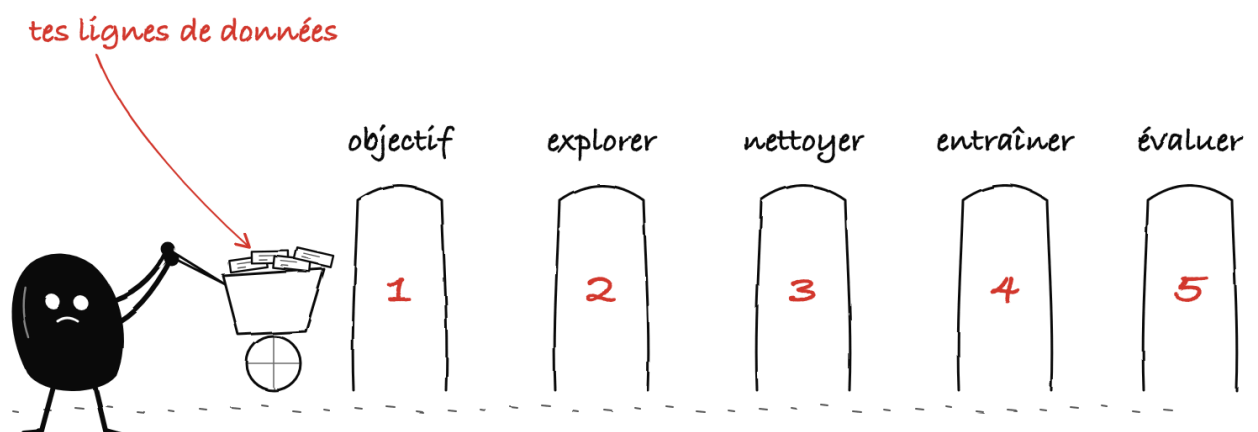
Apprentissage supervisé, classification, régression :
un cours en dessins, sans jargon et sans code.

Partie 1	Les bases et l'apprentissage supervisé
Partie 2	Préparer les données
Partie 3	La classification
Partie 4	La régression
Partie 5	Les pièges : surapprentissage et cie
Fin	Mini-lexique

Comment lire ce document ?

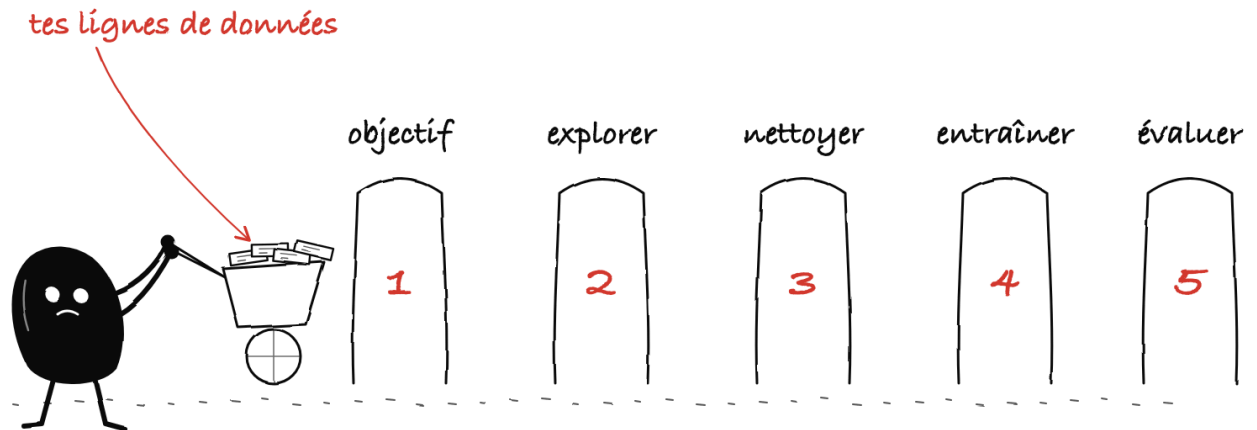
Chaque page = un dessin + l'idée en une phrase + une explication + une comparaison de la vie de tous les jours.

Le chemin d'un modèle



Le Machine Learning, c'est quoi ?

Le chemin d'un modèle



L'idée : Le **Machine Learning** (apprentissage automatique) consiste à apprendre à un ordinateur à faire des prédictions **à partir d'exemples**, au lieu de lui écrire toutes les règles à la main.

Xiaohei pousse une brouette pleine de fiches (les lignes de ton tableau de données) à travers 5 portiques, toujours dans le même ordre :

- **Objectif** : que veut-on prédire ? (ex. : un client va-t-il partir ?)
- **Explorer** : regarder les données pour les comprendre.
- **Nettoyer** : réparer les données abîmées (trous, valeurs bizarres, mots).
- **Entraîner** : laisser l'ordinateur apprendre sur des exemples.
- **Évaluer** : vérifier s'il prédit bien.

En image : Comme préparer une recette : choisir le plat, regarder les ingrédients, les laver et les couper, cuisiner, puis goûter.

L'apprentissage supervisé : apprendre avec les réponses

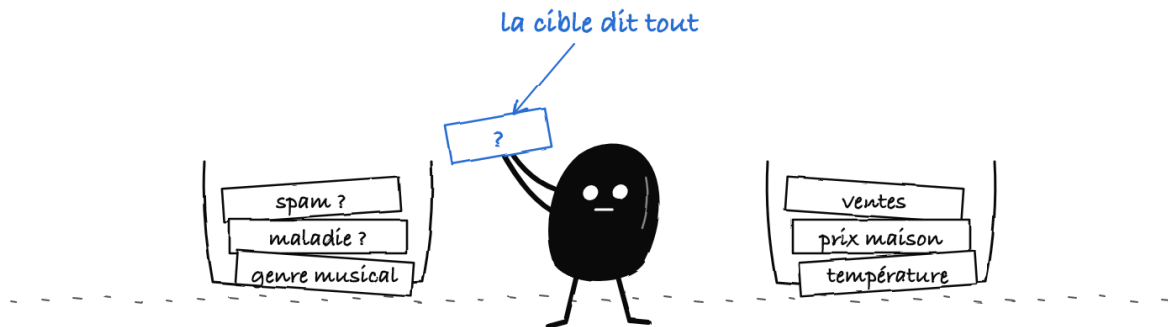
une catégorie ou un nombre ?

classification

→ une catégorie

régression

→ un nombre



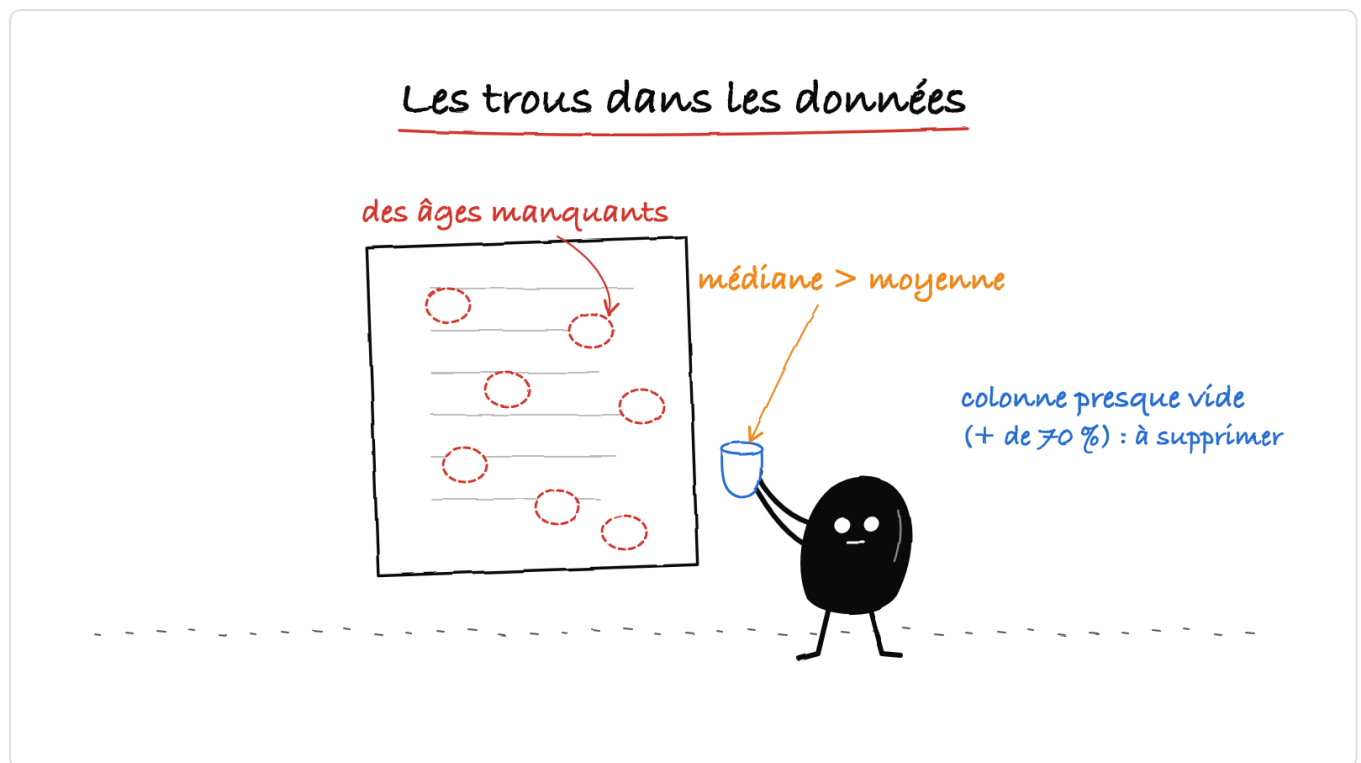
L'idée : En **apprentissage supervisé**, on montre à l'ordinateur des exemples **dont on connaît déjà la réponse**. Il apprend le lien entre les informations et la réponse, puis il devine la réponse pour de nouveaux cas.

Le vocabulaire à connaître :

- **Features** (caractéristiques) : les informations qui décrivent chaque cas (âge, taille, revenu...).
- **Cible** (target) : la réponse qu'on veut prédire.
- **Données d'entraînement** : les exemples pour apprendre. **Données de test** : les exemples gardés de côté pour vérifier.
- La **cible décide du type de tâche** : une **catégorie** (spam ou non, genre musical) = **classification** ; un **nombre** (ventes, prix, température) = **régression**.

En image : Deviner « est-ce un chat ou un chien ? » (catégorie) n'est pas la même chose que deviner « combien pèse ce chat ? » (nombre).

Les trous dans les données



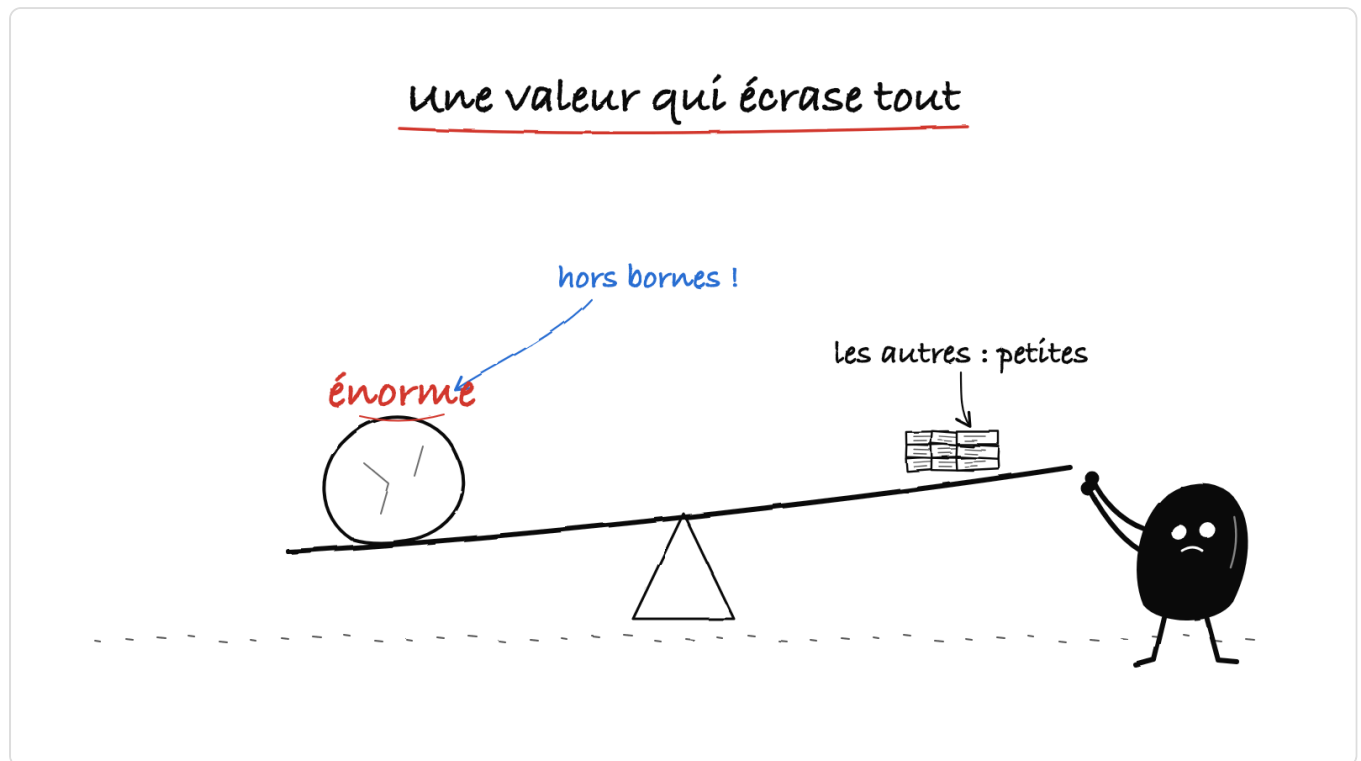
L'idée : Une **valeur manquante**, c'est une case vide dans le tableau. Un modèle ne sait pas calculer avec du vide.

Exemple : des âges manquent pour une partie des personnes, et une colonne est presque entièrement vide (plus de 70 %). Xiaohei bouche les trous avec une valeur de remplacement :

- **Un nombre (l'âge) :** on met la **médiane** (la valeur « du milieu » quand on range tout le monde du plus petit au plus grand).
- **Une catégorie (une ville, un type...) :** on met la valeur la plus fréquente (le « mode »).
- **Une colonne vide à plus de 70 % :** on la supprime, il n'y a presque rien à en tirer.

En image : Pourquoi la médiane plutôt que la moyenne ? Si un milliardaire entre dans un bar, la moyenne des fortunes explose, mais la médiane ne bouge presque pas.

Une valeur qui écrase tout



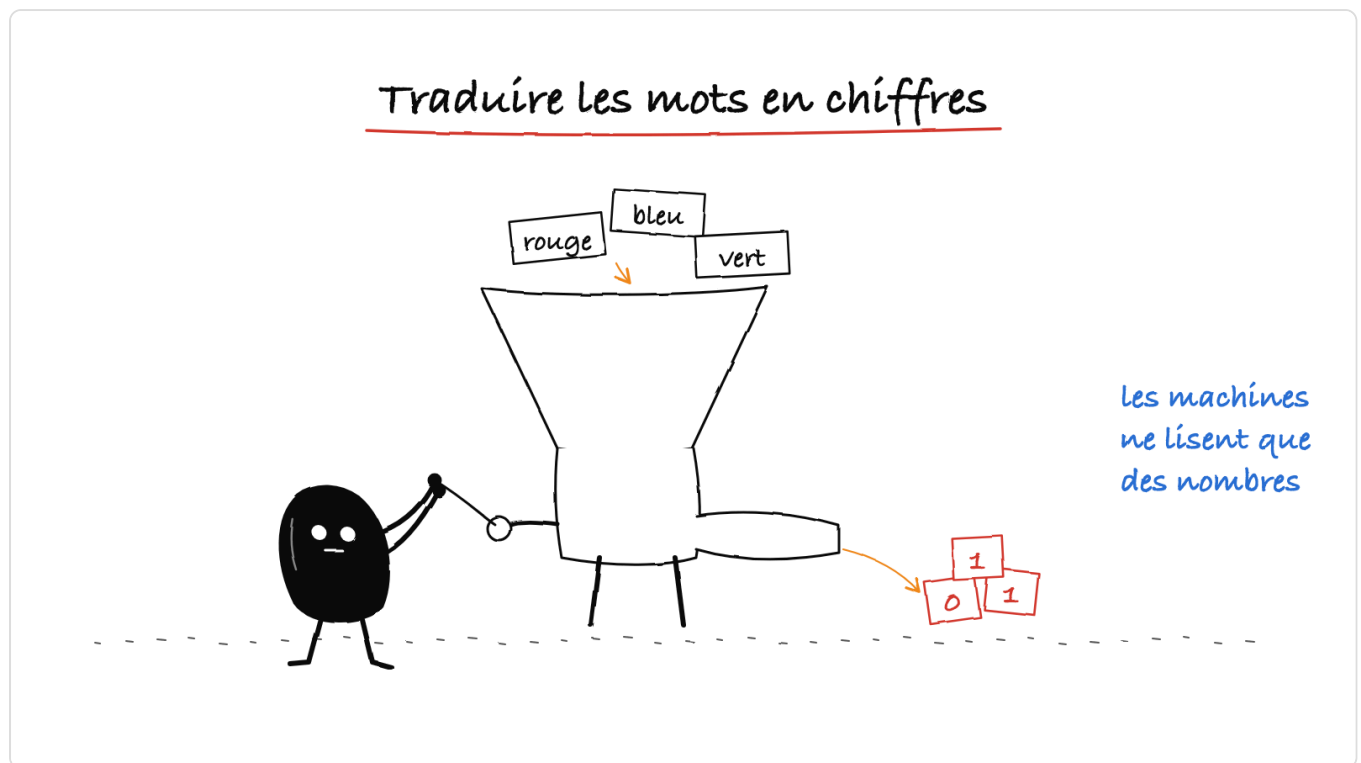
L'idée : Un **outlier** (valeur aberrante) est une valeur très éloignée de toutes les autres.

Exemple : un prix vaut des dizaines de fois le prix « du milieu » (la médiane). C'est un énorme rocher posé sur une balançoire de petites cartes. On le repère avec la méthode de l'**IQR** : on calcule des « bornes », et ce qui est au-delà est suspect.

- **Garder** si c'est une vraie information.
- **Supprimer** si c'est clairement une erreur.
- **Adoucir** avec le logarithme, qui rapproche les valeurs extrêmes des autres.

En image : Un élève a 20/20 quand tous les autres ont entre 5 et 8. On ne l'ignore pas, mais on évite qu'il fausse la moyenne de la classe.

Traduire les mots en chiffres



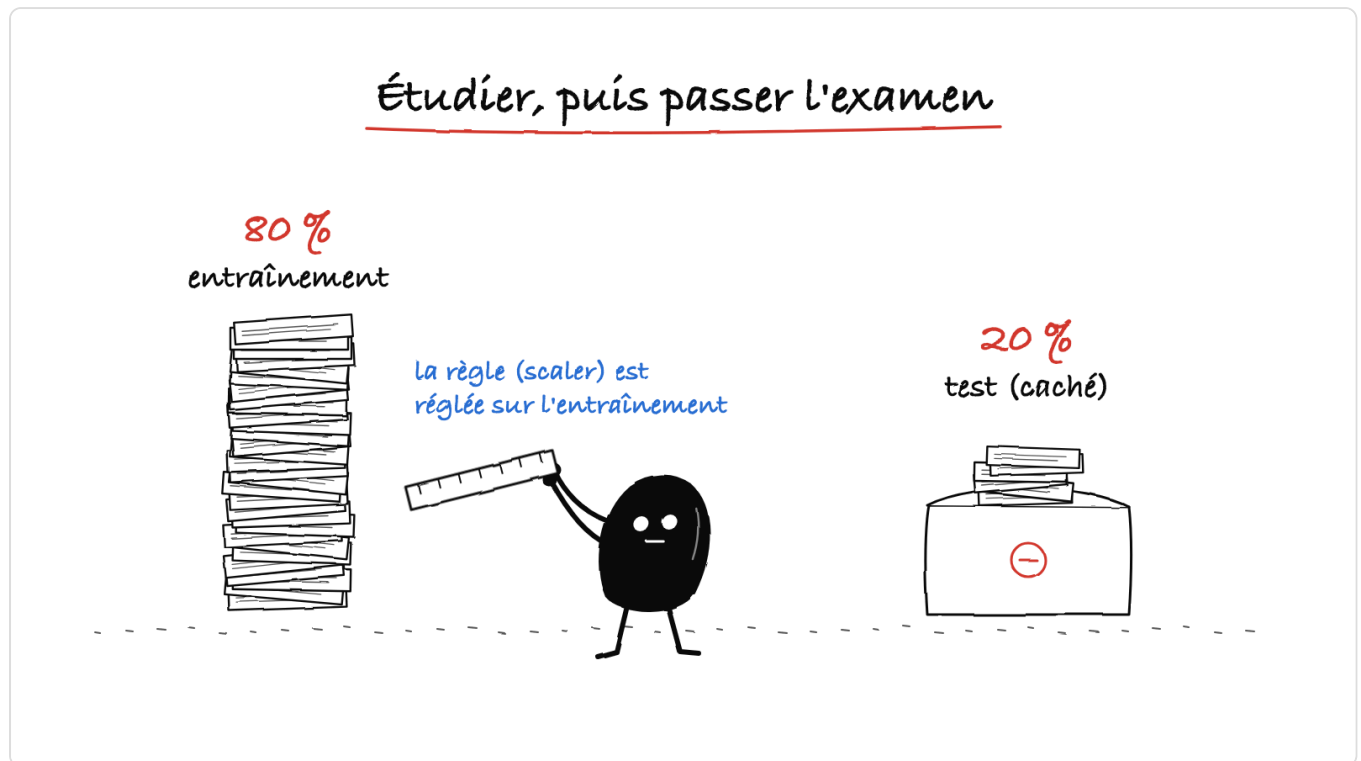
L'idée : L'**encodage** transforme les mots (« homme », « femme », un nom de ville...) en nombres.

Un ordinateur ne comprend que les nombres. Xiaohei tourne la manivelle d'un hachoir : les mots entrent, des 0 et des 1 sortent (c'est le « one-hot »). Deux autres idées utiles :

- **Standardisation** : on remet les colonnes à la même échelle. Sans cela, un prix (jusqu'à plusieurs centaines) écrase un âge (jusqu'à 80), simplement parce que ses nombres sont plus grands.
- **Créer de nouvelles colonnes utiles** (feature engineering) : par exemple « taille de la famille » = frères/sœurs/conjoint + parents/enfants.

En image : C'est comme traduire deux livres dans la même langue avant de les comparer.

Étudier, puis passer l'examen



L'idée : On coupe les données en deux paquets : **entraînement** (80 %) pour apprendre, **test** (20 %) pour vérifier.

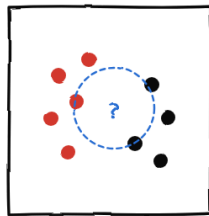
Exemple : 80 % des lignes pour s'entraîner, 20 % enfermées dans un coffre pour l'examen surprise. Xiaohei mesure avec une règle (le « scaler »), mais il ne la règle que sur le grand paquet.

- Si le modèle voyait le test, il pourrait juste **réciter par cœur**.
- **Fuite de données (data leakage)** : si la règle était réglée aussi sur le test, le modèle aurait un « aperçu » de l'examen et ses notes seraient trop belles.
- En code : `train_test_split(..., test_size=0.2)` , puis apprendre sur l'entraînement seulement.

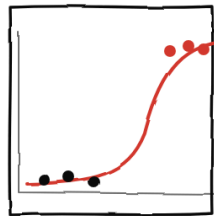
En image : Un élève qui a vu les questions avant aura 20/20 sans avoir compris. Le test doit rester une vraie surprise.

Cinq juges, un même cas

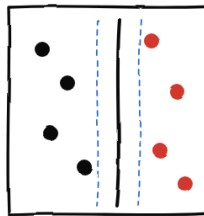
Cinq juges, un même cas



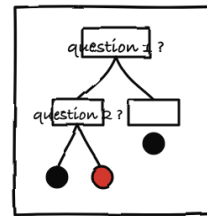
voisins (KNN)



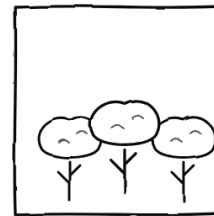
courbe en S



frontière (SVM)



arbre : oui / non



forêt = vote

chaque juge a sa méthode

cas ?

quelle catégorie ?



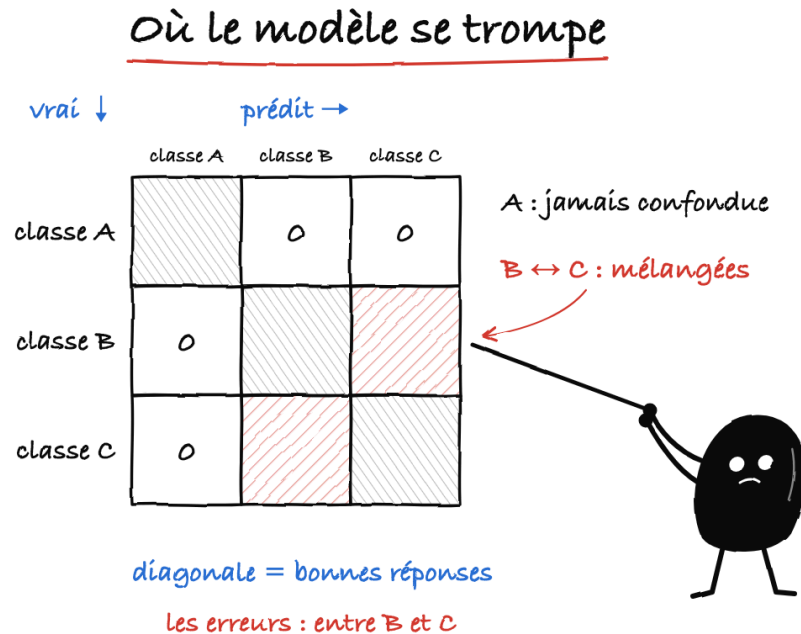
L'idée : La **classification** consiste à ranger chaque cas dans une catégorie (ex. : spam ou non, ou une catégorie parmi trois).

Xiaohei présente le même cas à cinq « juges » qui raisonnent chacun à leur façon :

- **KNN** : regarde les voisins les plus proches et vote.
- **Régression logistique** : calcule une probabilité, avec une courbe en S (entre 0 et 1).
- **SVM** : trace la frontière la plus large possible entre les groupes.
- **Arbre de décision** : pose des questions à la suite (« question 1 ? », « question 2 ? »).
- **Forêt aléatoire** : plein d'arbres qui votent ensemble, plus fiable qu'un seul.

En image : Demander l'avis de cinq médecins : chacun a sa méthode. Le bon réflexe : les comparer sur le même jeu de test.

Où le modèle se trompe



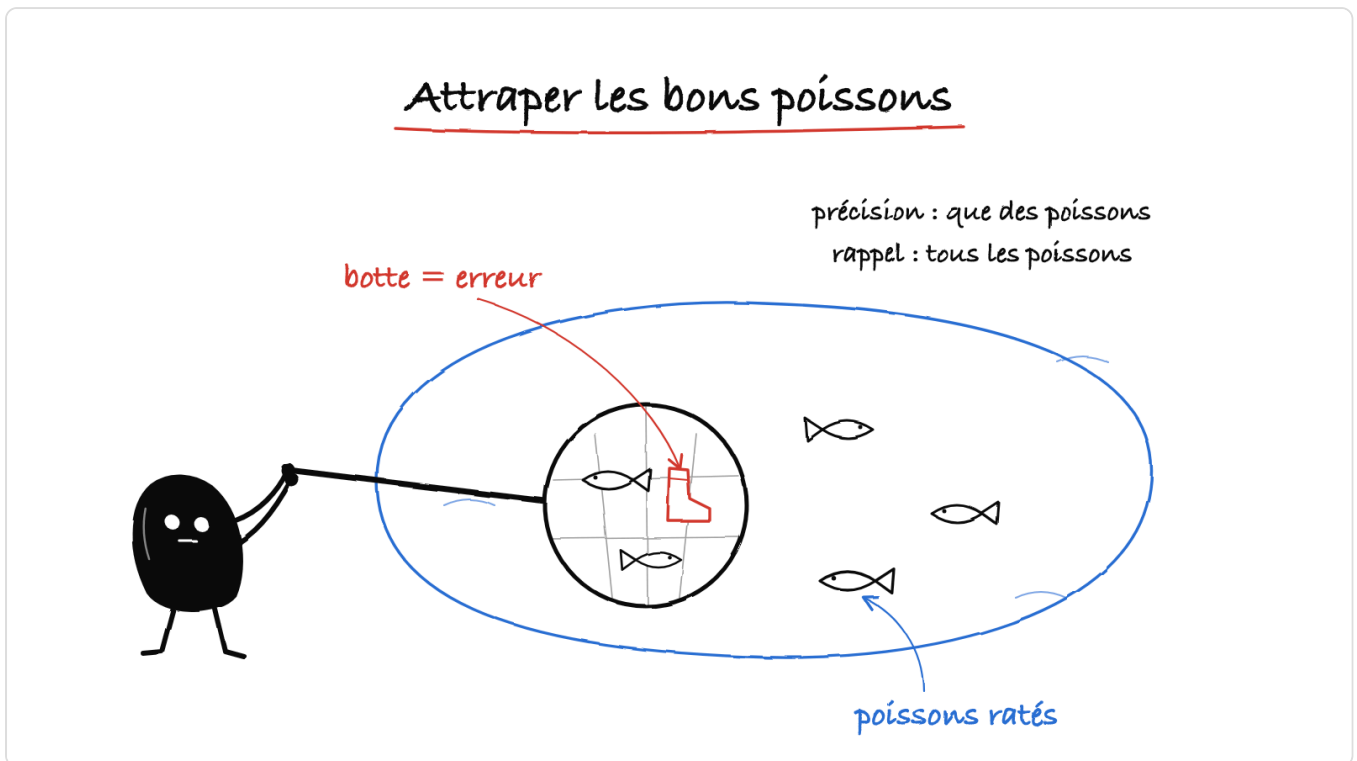
L'idée : La **matrice de confusion** est un tableau qui compare la vraie catégorie (lignes) à la catégorie prédite (colonnes).

La diagonale (grisée) contient les bonnes réponses. Tout ce qui est hors diagonale est une erreur.

- Une classe très différente des autres n'est presque jamais confondue.
- Les erreurs viennent des deux classes qui se ressemblent.
- Le meilleur modèle est celui qui a le moins d'erreurs hors diagonale ; on compare plusieurs modèles sur le même test.

En image : Comme un correcteur qui confond toujours les deux mêmes lettres qui se ressemblent, mais jamais les autres.

Attraper les bons poissons



L'idée : Pour juger un modèle de classification, on ne regarde pas seulement « il a raison X % du temps » (l'**accuracy**, fiable quand les catégories sont équilibrées).

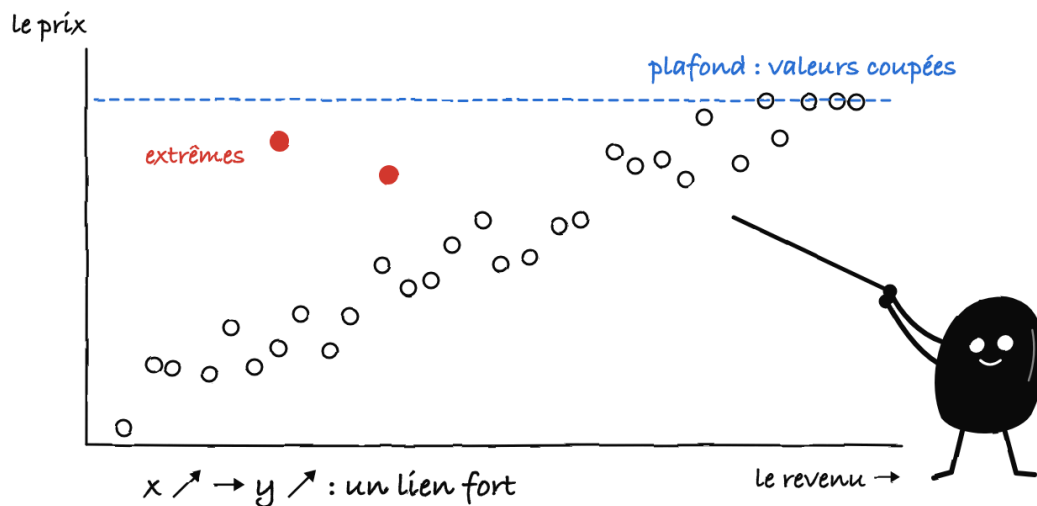
Xiaohei pêche avec un filet. Les poissons sont les vrais cas à trouver, les bottes sont les erreurs.

- **Précision** : dans mon filet, quelle part est de vrais poissons ? (peu de bottes = bonne précision)
- **Rappel** : parmi tous les poissons du lac, combien j'en ai attrapé ? (beaucoup de poissons ratés = mauvais rappel)
- **F1-score** : une note qui équilibre les deux.

En image : Un filet énorme attrape tous les poissons mais aussi plein de bottes. Un filet minuscule n'attrape que de bons poissons, mais très peu. Le bon modèle est entre les deux.

Régression : explorer avant de prédire

Explorer avant de prédire



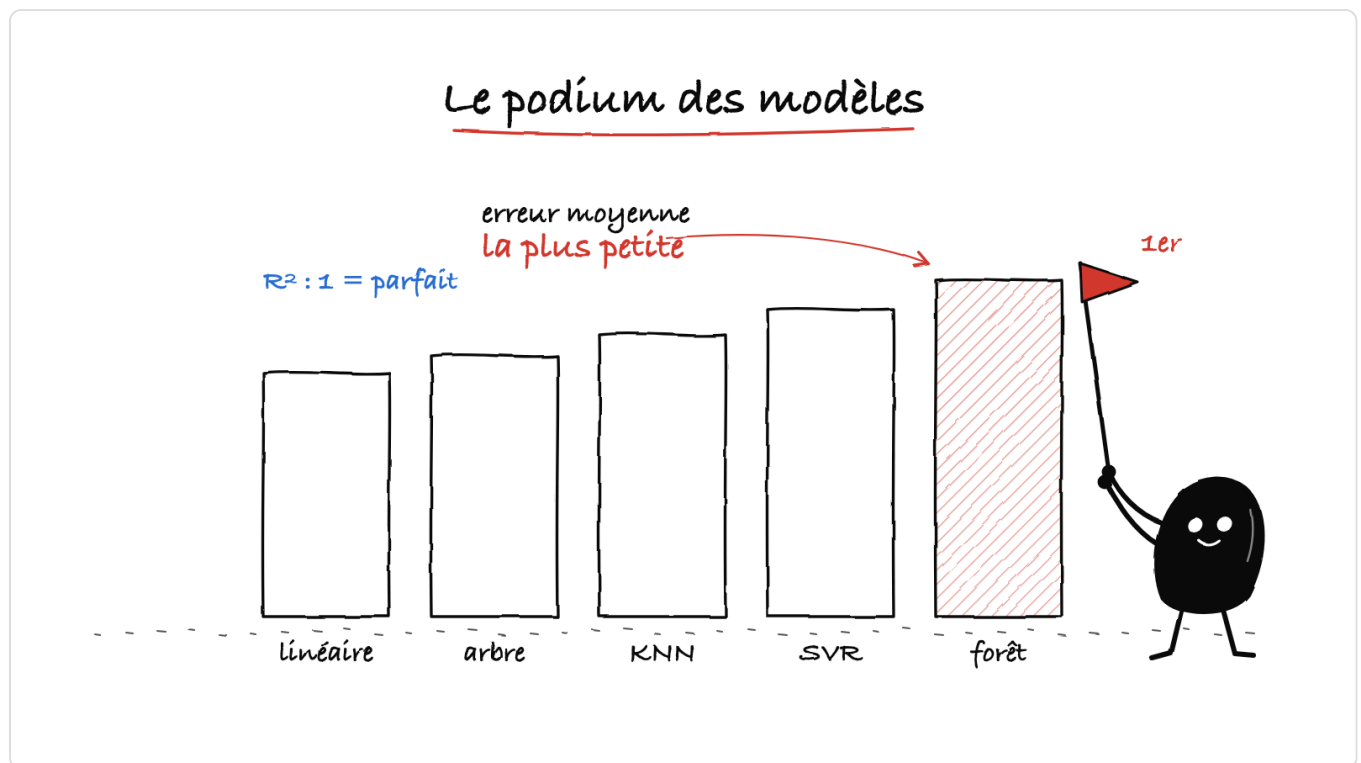
L'idée : La **régression** prédit un **nombre** (ici : le prix d'un logement). Avant de modéliser, on **explore** les données.

Exemple : un tableau de milliers de zones avec le prix d'un logement ; toutes les colonnes sont des nombres, aucune case vide. En traçant le prix selon le revenu, on voit tout de suite le motif :

- Plus le revenu est haut, plus le prix est haut : un **lien fort** (corrélation proche de 1).
- Le prix est **plafonné** : beaucoup de zones sont bloquées à la même valeur maximale (les valeurs sont « coupées »).
- Quelques **valeurs extrêmes** s'écartent du nuage de points.
- Les lieux comptent aussi : le prix varie de façon **non linéaire** selon la position (une simple droite ne suffit pas).

En image : Un nuage de points : d'un coup d'œil, on voit la tendance, le plafond et les valeurs qui dépassent.

Le podium des modèles de régression



L'idée : En régression on mesure l'**erreur** : plus elle est petite, mieux c'est.

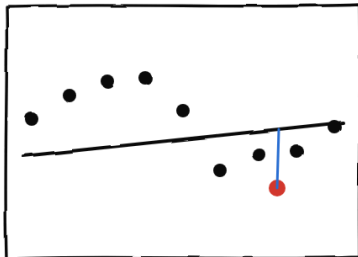
On compare 5 modèles sur les données de test avec trois notes :

- **MAE** : l'erreur moyenne. La forêt aléatoire a l'erreur la plus petite ; la régression linéaire la plus grande.
- **MSE** : pareil, mais les grosses erreurs comptent double. La régression linéaire en fait de très grosses (elle prédit même des prix négatifs).
- **R^2** : note de qualité, 1 = parfait. Classement obtenu : forêt, SVR, KNN, arbre, régression linéaire.
- Verdict : la **forêt aléatoire** gagne sur les trois notes. Mais elle est plus difficile à expliquer qu'une simple droite.

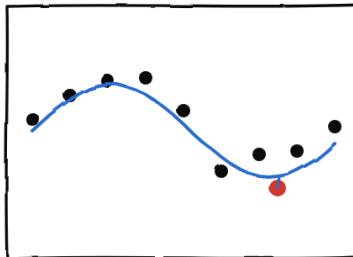
En image : Cinq apprentis devinent le prix de maisons : on compare à quel point chacun se trompe, en moyenne.

Le surapprentissage (et son contraire)

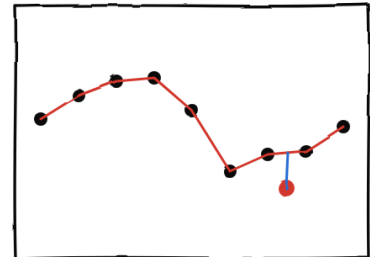
Trop simple, juste bien, trop compliqué



trop simple
sous-apprentissage



juste bien
le bon équilibre



par cœur
surapprentissage



point rouge = nouvelle donnée
trait bleu = l'erreur

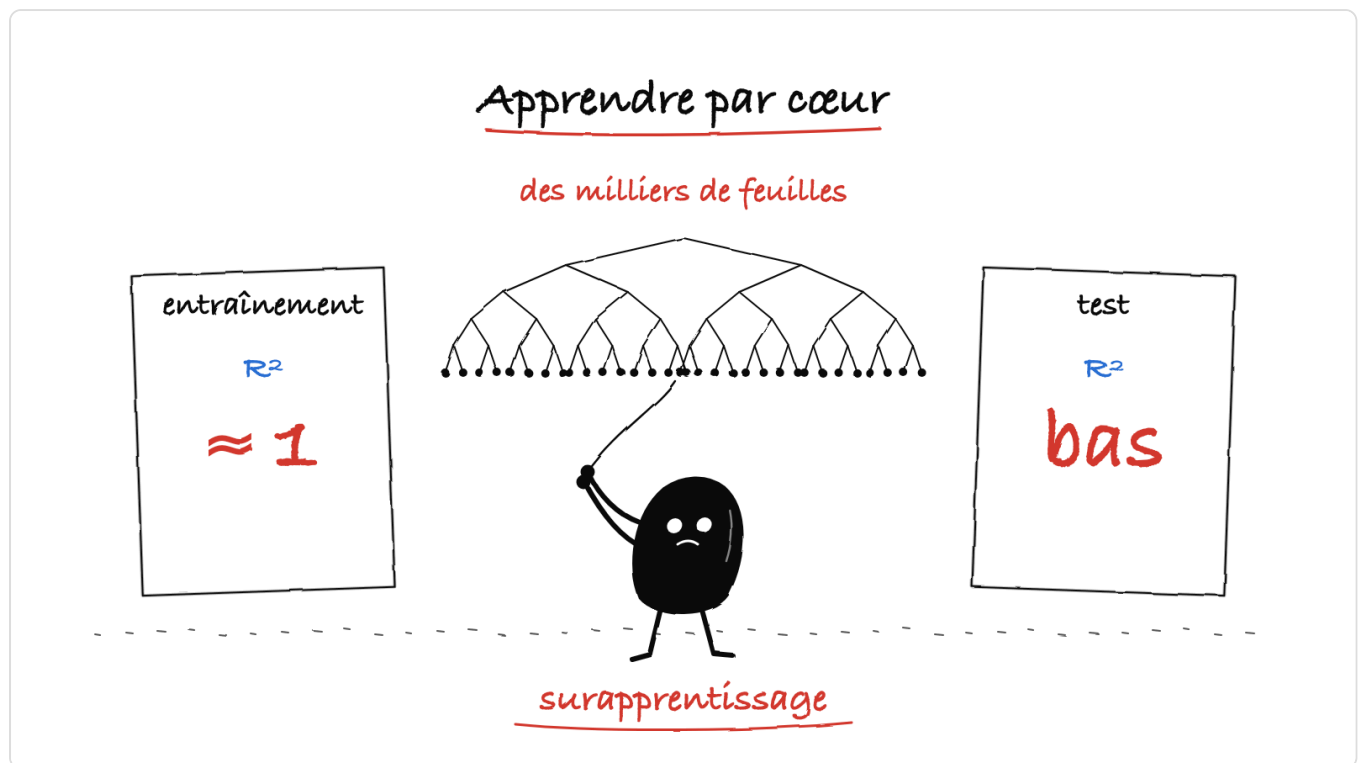
L'idée : Un bon modèle doit trouver la **règle générale** cachée dans les exemples, pas seulement les recopier.

Sur le dessin, les points noirs sont les exemples d'entraînement, le point rouge est un nouveau cas :

- **Sous-apprentissage** (à gauche) : le modèle est trop simple (ici une droite). Il se trompe partout, même sur l'entraînement.
- **Bon équilibre** (au milieu) : il suit la forme générale des données et prédit bien les nouveaux cas.
- **Surapprentissage** (à droite) : le modèle est trop compliqué. Il colle à chaque point d'entraînement, y compris au « bruit » (les petits hasards), et rate les nouveaux cas.
- **Comment le repérer ?** Note excellente à l'entraînement, note bien plus faible au test.
- **Comment l'éviter ?** Un modèle plus simple, plus de données, une forêt à la place d'un seul arbre, ou la validation croisée (tester sur plusieurs découpages).

En image : Un élève qui apprend les corrections par cœur réussit les exercices déjà vus, mais échoue dès que la question change.

Un exemple chiffré de surapprentissage



L'idée : Un arbre de décision très profond est un cas classique de surapprentissage.

Dans l'exemple des prix de logements, l'arbre est très profond, avec des milliers de feuilles. Il connaît parfaitement ses données d'entraînement, mais il est perdu face à de nouveaux cas.

- Note à l'entraînement : $R^2 \approx 1$ (parfait).
- Note au test : R^2 **bien plus bas**.
- Un grand écart entre les deux notes est le signal d'alerte.
- Autre piège de la classification : des **classes déséquilibrées**. Si 95 % des cas sont « non défaut », un modèle qui répond toujours « non défaut » a 95 % de bonnes réponses sans rien comprendre. On regarde alors le rappel et le F1-score.

En image : Un modèle n'est pas bon parce qu'il a une belle note à l'entraînement, mais parce qu'il réussit sur des cas qu'il n'a jamais vus.

Mini-lexique

Feature	une information qui décrit un cas (âge, prix, taille...)
Cible (target)	la réponse qu'on veut prédire
Entraînement / test	données pour apprendre / données gardées pour vérifier
Classification	prédire une catégorie
Régression	prédire un nombre
Valeur manquante	case vide dans le tableau
Outlier	valeur très éloignée des autres
Encodage	transformer des mots en nombres
Standardisation	mettre les colonnes à la même échelle
Accuracy	part de bonnes réponses
Précision / rappel / F1	fiabilité des « oui » / part des vrais cas trouvés / note qui équilibre les deux
MAE / MSE / R²	erreur moyenne / erreur qui punit les grosses erreurs / note de qualité (1 = parfait)
Surapprentissage	le modèle apprend par cœur et échoue sur du nouveau
Sous-apprentissage	le modèle est trop simple pour comprendre
Fuite de données	le test influence l'entraînement, ce qui truque les notes